

VERİ TEMİZLEME VE ETL SÜREÇLERİ TASARIM PROJESİ

1. Giriş ve Amaç

Bu çalışma, Kaggle platformundan temin edilmiş “Global Freelancers” veri setinin CSV dosyasından (kirli_data) PostgreSQL tabanlı bir veritabanı yapısına aktarılması ve analize hazır hale getirilmesi sürecini kapsamaktadır. Bu projenin temel amacı; veri tutarsızlıklarını ortadan kaldırmak, veriyi standart bir şemaya oturtmak ve eksik verileri mantıksal bir çerçevede tamamlayarak veri setinin analiz edilebilirliğini maksimum seviyeye çıkarmaktır.

2. ETL Mimarisi

Proje, veri bütünlüğünü korumak adına üç aşamalı bir katman mimarisi ile kurgulanmıştır:

- “global_freelancers_raw.csv” dosyası, “kirli_data” isimli geçici bir tabloya ham haliyle aktarılmıştır. Bu aşamada herhangi bir veri ile ilgili işlemler yapılmamış, verinin orijinal hali korunmuştur.
- “stage_data” tablosunda verinin temizlenmesi, dönüştürülmesi ve eksik değerlerin mantık çerçevesinde ataması işlemleri gerçekleştirilmiştir.
- Dönüştürülmüş ve doğrulanmış veriler, nihai sorguların çalıştırılacağı ve raporlamaların yapılacağı “temiz_data” tablosuna aktarılmıştır.

3. Tespit Edilen Veri Kalitesi Sorunları

Yapılan ön inceleme, veri setinde temel kalite sorunlarını ortaya koymuştur:

- Format Tutarsızlığı: “gender” sütunundaki kategorik değerlerin 10 farklı varyasyonda (f, Female, FEMALE, m, Male, m., vb.) girilmiş olması, gruplandırma işlemlerini imkansız kıldığı gözlemlenmiştir.
- Sembol Kirliliği: “hourly_rate (USD)” sütunundaki para birimi ibareleri (USD, \$) ve “client_satisfaction” sütunundaki “%” işaretleri, verinin string formatında kalmasına neden olmuş ve sayısal hesaplamaları engellediği gözlemlenmiştir.
- Eksik Veri Dağılımı:
 - client_satisfaction: 176 satır
 - rating: 101 satır
 - years_of_experience: 51 satır.
 - age: 30 satır.
- Mantık Karmaşası: “is_active” sütununda yer alan True, False, Yes, No, 1 ve 0 gibi çoklu format yapısı, veri setinde standart bir ikili sistemin eksikliğini göstermektedir.

4. Uygulanan Dönüştürme Mantığı

A. Veri Standardizasyonu ve Temizleme

- Kategorik Standardizasyon: CASE WHEN koşulları kullanılarak “gender” sütunu, sadece 'F' ve 'M' değerlerini içerecek şekilde normalize edilmiştir.
- Sayısal Dönüştürme: REPLACE ve CAST (veya TO_NUMBER) fonksiyonları kullanılarak “hourly_rate” ve “client_satisfaction” sütunları numeric veri tipine dönüştürülmüştür.
- Boolean Normalizasyonu: is_active sütunu, CASE yapısı ile 1 (Aktif) ve 0 (Pasif) olacak şekilde kodlanmıştır.

B. Eksik Veri Atama

Veri setinin doğasına uygun olarak aşağıdaki özel imputasyon stratejileri uygulanmıştır:

1. Deneyim Bazlı Gruplandırma: “years_of_experience” sütunundaki boşluklar, kullanıcıların yaş gruplarına göre ayrılarak(5'er yıllık yaş aralıkları) bu grupların ortalama değerleriyle doldurulmuştur. Bu yöntem, aykırı değerlerin ve yaş-tecrübe arasındaki gerçekleşebilecek olası mantık hatalarının önüne geçmeyi sağlamıştır.
2. Genel Ortalama: “age”, “hourly_rate” ve “client_satisfaction” gibi sayısal sütunlarda, boş değerler genel veri seti ortalaması ile doldurulmuştur.
3. Mantıksal Varsayım: “rating” sütunundaki eksiklikler, henüz bir değerlendirme almamış profil olarak yorumlanmış ve 0 değeri atanmıştır.
4. Mod Ataması: “is_active” sütununda, kategorik dağılımın bozulmaması için veri setindeki en baskın değer kullanılmıştır.

5. Veri Yükleme ve Kalite Kontrol Süreci

Sürecin sonunda, “stage_data” tablosu üzerinde gerçekleştirilen tüm işlemler “temiz_data” tablosuna aktarılmıştır. Gerçekleştirilen SQL kalite kontrol sorguları sonucunda, veri setindeki tüm eksik değerlerin giderildiği, veri tiplerinin analiz için doğru formatta olduğu doğrulanmıştır. Bu ETL süreci, verinin ham halinden arındırılıp karar destek sistemleri için güvenilir bir temel oluşturmasını sağlamıştır.

6. Yürütülen Sorgular ve Açıklamaları

Aşağıda görselleri bulunan resimlerde proje boyunca çalıştırılan veri temizleme ve atama işlemlerini kapsayan sorgular yer almaktadır.

Resim 1.

```
UPDATE stage_data
SET gender = CASE
  WHEN gender IN ('M', 'MALE', 'Male', 'male', 'm') THEN 'Male'
  WHEN gender IN ('F', 'FEMALE', 'Female', 'female', 'f') THEN 'Female'
  ELSE gender
END;

UPDATE stage_data
SET "hourly_rate (USD)" = REPLACE(REPLACE("hourly_rate (USD)", 'USD ', ''), '$', '');

UPDATE stage_data
SET is_active = CASE
  WHEN LOWER(is_active) IN ('1', 'true', 'yes', 'y') THEN '1'
  WHEN LOWER(is_active) IN ('0', 'false', 'no', 'n') THEN '0'
  ELSE is_active
END;

UPDATE stage_data
SET client_satisfaction = REPLACE(client_satisfaction, '%', '');
```

İlk sorguda, “gender” sütunundaki farklı yazım biçimleri standart hale getirilmiştir. “M, MALE, male, m” gibi varyasyonlar tek bir “Male” değerine; “F, FEMALE, female, f” gibi ifadeler ise “Female” değerine dönüştürülmüştür. Böylece kategorik veri tutarlılığı sağlanmış ve analizlerde hatalı gruplanmaların önüne geçilmiştir.

İkinci sorguda, "hourly_rate (USD)" sütunundaki para birimi ifadeleri temizlenmiştir. “USD ” ve “\$” gibi metinsel semboller kaldırılarak veri daha sade bir formata getirilmiştir. Bu işlem, ilgili sütunun daha sonra sayısal veri tipine çevrilmesini ve matematiksel hesaplamalarda kullanılmasını mümkün kılar.

Üçüncü sorguda, “is_active” sütunundaki farklı boolean ifadeler normalize edilmiştir. “true, yes, y, 1” gibi aktif durumu temsil eden değerler “1” olarak; “false, no, n, 0” gibi pasif durumu ifade eden değerler “0” olarak yeniden kodlanmıştır. Bu sayede veri setinde ikili bir yapı oluşturulmuştur.

Dördüncü sorguda ise “client_satisfaction” sütunundaki “%” sembolleri kaldırılmıştır. Bu temizlik işlemi, verinin string formattan çıkarılıp sayısal formata dönüştürülmesine olanak tanır ve ortalama gibi istatistiksel hesaplamaların yapılabilmesini sağlar.

Resim 2.

```
SELECT
  CASE
    WHEN age BETWEEN 20 AND 25 THEN '20-25'
    WHEN age BETWEEN 26 AND 30 THEN '26-30'
    WHEN age BETWEEN 31 AND 35 THEN '31-35'
    WHEN age BETWEEN 36 AND 40 THEN '36-40'
    WHEN age BETWEEN 41 AND 45 THEN '41-45'
    WHEN age BETWEEN 46 AND 50 THEN '46-50'
    WHEN age BETWEEN 51 AND 55 THEN '51-55'
    WHEN age BETWEEN 56 AND 60 THEN '56-60'
    ELSE 'Diğer'
  END AS yas_grubu,
  ROUND(AVG(years_of_experience::numeric), 1) AS ortalama_tecrube,
  COUNT(*) AS kisi_sayisi
FROM stage_data
GROUP BY yas_grubu
ORDER BY yas_grubu;

UPDATE stage_data k
SET years_of_experience = (
  SELECT ROUND(AVG(sub.years_of_experience::numeric), 0)
  FROM stage_data sub
  WHERE (
    CASE
      WHEN sub.age BETWEEN 20 AND 25 THEN '20-25'
      WHEN sub.age BETWEEN 26 AND 30 THEN '26-30'
      WHEN sub.age BETWEEN 31 AND 35 THEN '31-35'
      WHEN sub.age BETWEEN 36 AND 40 THEN '36-40'
      WHEN sub.age BETWEEN 41 AND 45 THEN '41-45'
      WHEN sub.age BETWEEN 46 AND 50 THEN '46-50'
      WHEN sub.age BETWEEN 51 AND 55 THEN '51-55'
      WHEN sub.age BETWEEN 56 AND 60 THEN '56-60'
    END
  ) = (
    CASE
      WHEN k.age BETWEEN 20 AND 25 THEN '20-25'
      WHEN k.age BETWEEN 26 AND 30 THEN '26-30'
      WHEN k.age BETWEEN 31 AND 35 THEN '31-35'
      WHEN k.age BETWEEN 36 AND 40 THEN '36-40'
      WHEN k.age BETWEEN 41 AND 45 THEN '41-45'
      WHEN k.age BETWEEN 46 AND 50 THEN '46-50'
      WHEN k.age BETWEEN 51 AND 55 THEN '51-55'
      WHEN k.age BETWEEN 56 AND 60 THEN '56-60'
    END
  )
  AND sub.years_of_experience IS NOT NULL
)
WHERE k.years_of_experience IS NULL
AND k.age IS NOT NULL;
```

İlk sorguda, “age” değişkeni belirli aralıklara bölünerek yaş grupları (örneğin 20–25, 26–30 vb.) oluşturulmuştur. Her bir yaş grubu için “years_of_experience” değerlerinin ortalaması hesaplanmış ve aynı zamanda o gruptaki kişi sayısı belirlenmiştir. Bu analiz, yaş ile deneyim arasındaki genel ilişkiyi görmek ve veri setinde anlamlı sonuçlar oluşturup mantıksal hataların önüne geçmek amacıyla yapılmıştır.

İkinci sorguda ise, “years_of_experience” sütunundaki eksik değerler doldurulmuştur. Her bir kaydın yaşına göre ait olduğu yaş grubu belirlenmiş ve o gruptaki mevcut kullanıcıların ortalama deneyim süresi hesaplanarak eksik değerlerin yerine atanmıştır. Bu yöntem sayesinde, rastgele bir değer atamak yerine daha mantıklı ve veriyle uyumlu bir imputasyon gerçekleştirilmiş, böylece veri bütünlüğü korunmuştur.

Resim 3.

```
UPDATE stage_data
SET "hourly_rate (USD)" = (
  SELECT ROUND(AVG("hourly_rate (USD)"::numeric), 2)::text
  FROM stage_data
  WHERE "hourly_rate (USD)" IS NOT NULL AND "hourly_rate (USD)" <> ''
)
WHERE "hourly_rate (USD)" IS NULL OR "hourly_rate (USD)" = '';

UPDATE stage_data
SET age = (SELECT ROUND(AVG(age::numeric), 0) FROM stage_data WHERE age IS NOT NULL)
WHERE age IS NULL;

UPDATE stage_data
SET rating = 0
WHERE rating IS NULL;

UPDATE stage_data
SET is_active = (
  SELECT is_active
  FROM stage_data
  WHERE is_active IS NOT NULL AND is_active <> ''
  GROUP BY is_active
  ORDER BY COUNT(*) DESC
  LIMIT 1
)
WHERE is_active IS NULL OR is_active = '';

UPDATE stage_data
SET client_satisfaction = (
  SELECT ROUND(AVG(client_satisfaction::numeric), 0)::text
  FROM stage_data
  WHERE client_satisfaction IS NOT NULL AND client_satisfaction <> ''
)
WHERE client_satisfaction IS NULL OR client_satisfaction = '';
```

İlk sorguda, "hourly_rate (USD)" sütunundaki eksik veya boş değerler doldurulmuştur. Bunun için veri setindeki mevcut saatlik ücretlerin ortalaması alınmış ve bu değer eksik kayıtlara atanmıştır. Böylece sütundaki veri kaybı giderilmiş ve analizlerde kullanılabilir hale getirilmiştir.

İkinci sorguda, "age" sütunundaki NULL değerler veri setinin genel yaş ortalaması ile doldurulmuştur. Bu yaklaşım, yaş bilgisinin tamamen kaybolmasını önlerken veri setinin genel dağılımını da büyük ölçüde korumayı amaçlar.

Üçüncü sorguda, "rating" sütununda eksik olan değerler 0 olarak atanmıştır. Bu, ilgili kullanıcıların henüz bir değerlendirme almadığı varsayımına dayanır ve eksik veri ile gerçek "0 puan" durumunun aynı şekilde ele alınmasını sağlar.

Dördüncü sorguda, "is_active" sütunundaki boş veya NULL değerler veri setinde en sık görülen değer ile yani mode hesaplanarak doldurulmuştur. Bu işlem, kategorik dağılımın bozulmasını önler ve veri setindeki genel aktif/pasif dengesini korur.

Beşinci sorguda ise, "client_satisfaction" sütunundaki eksik değerler ortalama memnuniyet oranı ile doldurulmuştur. Mevcut değerlerin ortalaması alınarak boş kayıtlara atanmış ve böylece sütunun sayısal analizlere uygun hale gelmesi sağlanmıştır.

VERİTABANI YEDEKLEME VE FELAKETTEN KURTARMA PLANI PROJESİ

1. Giriş ve Amaç

Bu rapor, PostgreSQL veritabanı yönetim sistemi kullanılarak gerçekleştirilen yedekleme ve felaketten kurtarma uygulamalarını içermektedir. Proje kapsamında tam yedekleme, fark yedekleme, otomatik zamanlayıcı kurulumu, felaketten kurtarma senaryosu ve yedek doğrulama testleri uygulanmıştır.

2. Veritabanı Yapısı

Proje kapsamında okul_db adında bir veritabanı oluşturulmuştur. Bu veritabanı içinde öğrenciler adlı bir tablo bulunmaktadır. Tablo; id, ad, soyad ve not_ortalama alanlarından oluşmaktadır. Tabloya başlangıçta Ali Yılmaz, Ayşe Kaya, Mehmet Demir, Zeynep Çelik ve Can Arslan olmak üzere 5 öğrenci kaydı eklenmiştir.

3. Yedekleme Stratejileri

3.1 Tam Yedekleme

Tam yedekleme, veritabanındaki tüm verilerin eksiksiz olarak yedeklenmesi işlemidir. Bu projede “yedekle.sh” adlı bir shell script yazılmıştır. Script çalıştırıldığında pg_dump aracı ile okul_db veritabanının tamamı yedeklenmektedir. Dosya adına otomatik olarak tarih ve saat eklenmektedir. Bu sayede her yedek ayrı bir dosyaya kaydedilmekte ve üzerine yazılma riski ortadan kalkmaktadır.

```
#!/bin/bash
export PATH=$PATH:/Library/PostgreSQL/18/bin
TARİH=$(date +%Y%m%d_%H%M)

pg_dump -U postgres -d okul_db -F c -f ~/Desktop/ATPDS_2/yedekler/tam_yedek_$TARİH.backup

echo "Yedek alındı: tam_yedek_$TARİH.backup"
```

3.2 Fark Yedekleme

Fark yedekleme, son tam yedekten bu yana değişen verilerin yedeklenmesi işlemidir. Tam yedeğe kıyasla daha az yer kaplar ve daha hızlı alınır. Bu projede “fark_yedek.sh” adlı script yazılmıştır. Script, pg_dump aracının -t parametresi ile sadece öğrenciler tablosunu yedeklemektedir. Tam yedek tüm veritabanını kapsarken fark yedek yalnızca değişen tabloyu kapsamaktadır. Bu durum dosya boyutlarının karşılaştırılmasıyla açıkça görülmektedir.

```
#!/bin/bash

TARİH=$(date +%Y%m%d_%H%M)

pg_dump -U postgres -d okul_db \
-F c \
-t ogrenciler \
-f ~/Desktop/ATPDS_2/yedekler/fark_yedek_${TARİH}.backup

echo "Fark yedek tamamlandı: fark_yedek_${TARİH}.backup"

echo "Mevcut yedekler:"
ls ~/Desktop/ATPDS_2/yedekler/
```

4. Otomatik Zamanlayıcı

Yedekleme işlemlerinin manuel olarak yapılması insan hatasına açıktır. Bu nedenle Mac işletim sisteminin crontab aracı kullanılarak yedekleme işlemi otomatik hale getirilmiştir. Her gün saat 15:59'da yedekle.sh scripti otomatik olarak çalışacak şekilde ayarlanmıştır. İşlemin sonucu log.txt dosyasına kaydedilmekte, böylece yedekleme geçmişi takip edilebilmektedir.

```
59 15 * * * ~/Desktop/ATPDS_2/yedekle.sh >> ~/Desktop/ATPDS_2/yedekler/log.txt 2>&1
```

5. Felaketten Kurtarma Senaryosu

Felaketten kurtarma senaryosunda önce ogrenciler tablosu DROP TABLE komutu ile silinmiştir. Tablonun silindiği SELECT komutu ile doğrulanmıştır. Ardından pg_restore aracı kullanılarak daha önce alınan tam yedek geri yüklenmiştir. Geri yükleme işleminin ardından tablonun ve tüm verilerin eksiksiz olarak geri geldiği doğrulanmıştır. Bu senaryo, kazara veri kaybı yaşandığında yedekten geri dönüşün mümkün olduğunu kanıtlamaktadır.

```
#!/bin/bash

echo "ogrenciler tablosu siliniyor"
psql -U postgres -d okul_db -c "DROP TABLE IF EXISTS ogrenciler;"
psql -U postgres -d okul_db -c "SELECT * FROM ogrenciler;"
echo "yedek geri yükleniyor"
pg_restore -U postgres -d okul_db -F c ~/Desktop/ATPDS_2/yedekler/tam_yedek_20260419_1405.backup
psql -U postgres -d okul_db -c "SELECT * FROM ogrenciler;"
```

6. Yedek Doğrulama Testi

Yedeklerin gerçekten çalışıp çalışmadığını test etmek için “yedek_test.sh” adlı script yazılmıştır. Bu script önce okul_db_test adında boş bir test veritabanı oluşturmaktadır. Ardından mevcut yedeği bu test veritabanına yüklemektedir. Son olarak orijinal veritabanı ile test veritabanındaki kayıt sayıları karşılaştırılmaktadır. Her iki veritabanında da 5 kayıt bulunduğu doğrulanmış ve yedeklerin sağlıklı olduğu kanıtlanmıştır. Bu test yöntemi asıl veritabanına hiç dokunmadan gerçekleştirilmektedir.

```
#!/bin/bash
echo "Test veritabanı oluşturuluyor"
psql -U postgres -c "DROP DATABASE IF EXISTS okul_db_test;"
psql -U postgres -c "CREATE DATABASE okul_db_test;"

echo "Yedek test veritabanına yükleniyor"
pg_restore -U postgres -d okul_db_test -F c ~/Desktop/ATPDS_2/yedekler/tam_yedek_20260419_1405.backup

echo "Orijinal veritabanı:"
psql -U postgres -d okul_db -c "SELECT count(*) FROM ogrenciler;"

echo "Test veritabanı:"
psql -U postgres -d okul_db_test -c "SELECT count(*) FROM ogrenciler;"

echo "Test tamamlandı!"
```

7. Sonuç

Bu proje kapsamında PostgreSQL veritabanı için eksiksiz bir yedekleme ve felaketten kurtarma planı oluşturulmuştur. Tam yedekleme ve fark yedekleme stratejileri uygulanmış, yedekler otomatik zamanlayıcıya bağlanmış, felaketten kurtarma senaryosu başarıyla test edilmiş ve yedeklerin doğruluğu ayrı bir test veritabanı üzerinde doğrulanmıştır. Bu sayede olası bir veri kaybı durumunda sistemin hızla eski haline getirilebileceği gösterilmiştir.

